

# Sensing & Estimation in Robotics Project 3

Rick Ho, PID: A69030974

## I. INTRODUCTION

The goal of this project is to implement simultaneous localization and mapping (visual SLAM) via the extended Kalman filter (EKF). Given the measurement from IMU and stereo camera, we can approximate the robot odometry and map the visual features back to world frame. With the visual features (landmarks), we can perform visual SLAM to enhance the accuracy of robot odometry.

## II. PROBLEM FORMULATION

### A. IMU localization via EKF prediction

In the first step, we utilize IMU measurement to perform the EKF prediction step. In the EKF prediction step, we use the motion model to predict the next robot state. Also we could use Taylor expansion to calculate the covariance matrix of state and noise. The motion model can be express as:

$$\mu_{t+1|t} = \mu_{t|t} \exp(\tau_t \hat{\mathbf{u}}_t) \quad (1)$$

$$\delta \mu_{t+1|t} = \exp(-\tau_t \hat{\mathbf{u}}_t) \delta \mu_{t|t} + \mathbf{w}_t \quad (2)$$

The prediction step with noise  $w_t \sim N(0, W)$

$$\mu_{t+1|t} = \mu_{t|t} \exp(\tau_t \hat{\mathbf{u}}_t) \quad (3)$$

$$\Sigma_{t+1|t} = \exp(-\tau_t \hat{\mathbf{u}}_t) \Sigma_{t|t} \exp(-\tau_t \hat{\mathbf{u}}_t)^T + W \quad (4)$$

where  $\hat{a}$  is curly hat of  $a$ , and  $\mathbf{u}_t = \begin{bmatrix} \mathbf{v}_t \\ \omega_t \end{bmatrix} \in \mathbb{R}^6$ ,

$$\hat{\mathbf{u}}_t = \begin{bmatrix} \hat{\omega}_t & \mathbf{v}_t \\ 0^T & 0 \end{bmatrix} \in \mathbb{R}^{4 \times 4}, \hat{\mathbf{u}} = \begin{bmatrix} \hat{\omega}_t & \hat{\mathbf{v}}_t \\ 0^T & \hat{\omega}_t \end{bmatrix} \in \mathbb{R}^{6 \times 6}$$

### B. Landmark mapping via EKF update

In this part, we will use the robot state obtained from previous step to map the visual features to world frame. Observational model for stereo camera is:

$$\mathbf{z}_{t,i} = K_s \pi({}_O T_I T_t^{-1} \underline{\mathbf{m}}_j) + \mathbf{v}_{t,i} \quad (5)$$

where  $\mathbf{v}_{t,i} \sim N(0, V)$  and  $\underline{\mathbf{a}}_j := \begin{bmatrix} \mathbf{a}_j \\ 1 \end{bmatrix}$ ,  $\pi(\mathbf{q}) := \frac{1}{q_3} \mathbf{q} \in \mathbb{R}^4$

Plugging in the observation model to EKF prediction step,

$$K_{t+1} = \Sigma_t H_{t+1}^T (H_{t+1} \Sigma_t H_{t+1}^T + I \otimes V)^{-1} \quad (6)$$

$$\mu_{t+1} = \mu_t + K_{t+1} (\mathbf{z}_{t+1} - K_s \pi({}_O T_I T_{t+1}^{-1} \underline{\mu}_t)) \quad (7)$$

$$\Sigma_{t+1} = (I - K_{t+1} H_{t+1}) \Sigma_t \quad (8)$$

where

$$I \otimes V := \begin{bmatrix} V & & \\ & \ddots & \\ & & V \end{bmatrix} \quad (9)$$

we can optimize the coordinate of visual features through every observation (images taken by stereo camera).

### C. Visual-inertial SLAM

Last part, we will use visual features to optimize the odometry we got in the first part. In this part, we will calculate the next state via EKF update step followed by prediction step. The prediction step in this part can be expressed as:

$$K_{t+1} = \Sigma_t H_{t+1}^T (H_{t+1} \Sigma_t H_{t+1}^T + I \otimes V)^{-1} \quad (10)$$

$$\mu_{t+1|t+1} = \mu_{t+1|t} \exp((K_{t+1} (\mathbf{z}_{t+1} - \tilde{\mathbf{z}}_{t+1}))^\wedge) \quad (11)$$

$$\Sigma_{t+1|t+1} = (I - K_{t+1} H_{t+1}) \Sigma_{t+1|t} \quad (12)$$

where  $\tilde{\mathbf{z}}_{t+1} := K_s \pi({}_O T_I \mu_{t+1|t}^{-1} \underline{\mathbf{m}}_j)$

By adjusting the noise ratio between motion model (W) and observation model (V), we can finally enhance the accuracy of the odometry.

## III. TECHNICAL APPROACH

### A. IMU localization via EKF prediction

To get the odometry of the robot, what we only need is the  $\mu_{t+1|t}$  from prediction step. Since we are not performing update step in this part, I skipped calculating the covariance of the robot state. Also, there is no need to add the noise term in this step.

First, I stacked the velocity and angular velocity to form generalized velocity  $\mathbf{u}_t$ . Next, transforming  $\mathbf{u}_t$  to pose matrix using formula below:

$$T_{k+1} = T_k \exp(\tau_t \hat{\mathbf{u}}_t) \quad (13)$$

where  $\exp(\hat{\xi})$  defined as:

$$\exp(\hat{\xi}) = I + \hat{\xi} + \left( \frac{1 - \cos\|\theta\|}{\|\theta\|^2} \right) \hat{\xi}^2 + \left( \frac{\|\theta\| - \sin\|\theta\|}{\|\theta\|^3} \right) \hat{\xi}^3 \quad (14)$$

### B. Landmark mapping via EKF update

In this part, we want to estimate the coordinates of visual features, so we need to calculate the derivative term ( $H$ ) corresponding to  $\underline{\mathbf{m}}$ . By deriving equation(5) corresponding to  $\underline{\mathbf{m}}$ , we get the derivative term

$$H_{t+1,i,j} = \begin{cases} K_s \frac{d\pi}{d\mathbf{q}}({}_O T_I T_{t+1}^{-1} \underline{\mu}_{t,j}) {}_O T_I T_{t+1}^{-1} P^T & \text{if } \Delta_t(j) = i \\ \mathbf{0} & \text{otherwise} \end{cases} \quad (15)$$

where

$$P^T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \text{ and } \frac{d\pi}{d\mathbf{q}}(\mathbf{q}) = \frac{1}{q_3} \begin{bmatrix} 1 & 0 & -\frac{q_1}{q_3} & 0 \\ 0 & 1 & -\frac{q_2}{q_3} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -\frac{q_4}{q_3} & 1 \end{bmatrix}$$

Note that the data provided has been processed, so the visual features given have been labeled. Hence, we will assign  $H$  to the block  $H_{t+1,i,i}$ . The  $H_{t+1} \in \mathbb{R}^{4N_t \times 3M}$ , where  $N_t$  is the number of visual features observed in time  $t$ , and  $M$  is the number of all visual features.

Before we start proceeding update step, we need to figure out how to perform triangulation. Given the intrinsic matrix  $K_l$  and  $K_r$ , we can "normalized" the image pixel by the camera model:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \frac{1}{Z_o} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X_o \\ Y_o \\ Z_o \\ 1 \end{bmatrix} \quad (16)$$

Multiplying  $inv(K)$  on both sides, we are able to translate left and right camera intrinsic calibration matrix both to  $I$ . Next, with stereo camera model:

$$\begin{bmatrix} u_L \\ v_L \\ u_R \\ v_R \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & -b \\ 0 & 1 & 0 & 0 \end{bmatrix} \frac{1}{z} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \quad (17)$$

we can compute the coordinate in optical frame through the following equation:

$$z = \frac{b}{u_L - u_R} \quad (18)$$

$$x = u_L z \quad (19)$$

$$y = v_L z \quad (20)$$

To transfer the coordinate of optical frame to IMU frame, we multiply  ${}_{IMU}T_C {}_CT_O$  to the coordinate of optical.

Because we need to estimate all the visual features, if the number of visual feature ( $M$ ) too large, it would cause slow computation speed. Therefore, I only keep  $\frac{1}{5}$  features for the SLAM. Also, to speed up my computation and to save memory, I used the `csr_matrix` in `scipy` package to store my matrix  $H \in \mathbb{R}^{4N_t \times 3M}$ ,  $K \in \mathbb{R}^{3M \times 4N_t}$  and  $\Sigma \in \mathbb{R}^{3M \times 3M}$ .

### C. Visual-inertial SLAM

Following the EKF prediction step (equation (1),(2)) and the EKF update step(equation (10)~(12)), we need to calculate derivative term  $H$  corresponding to  $T$ . We will get

$$H = -K_s \frac{d\pi}{d\mathbf{q}} ({}^OT_I \mu_{t+1|t}^{-1} \underline{\mathbf{m}}_j) {}^OT_I (\mu_{t+1|t}^{-1} \underline{\mathbf{m}}_j)^\odot \quad (21)$$

where

$$\begin{bmatrix} \mathbf{s} \\ 1 \end{bmatrix}^\odot := \begin{bmatrix} I & -\hat{\mathbf{s}} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{4 \times 6}$$

and the  $exp(\hat{\mathbf{u}})$  is defined as:

$$\begin{aligned} exp(\hat{\xi}) &= I + \left( \frac{3\sin|\theta| - |\theta||\cos|\theta||}{2|\theta|} \right) \hat{\xi}^1 + \\ &\left( \frac{4 - |\theta||\sin|\theta| - 4\cos|\theta||}{2|\theta|^2} \right) \hat{\xi}^2 + \left( \frac{\sin|\theta| - |\theta||\cos|\theta||}{2|\theta|^3} \right) \hat{\xi}^3 + \\ &\left( \frac{2 - |\theta||\sin|\theta| - 2\cos|\theta||}{2|\theta|^4} \right) \hat{\xi}^4 \quad (22) \end{aligned}$$

## IV. RESULTS

### A. Dataset 00

The odometry generated from EKF prediction step:

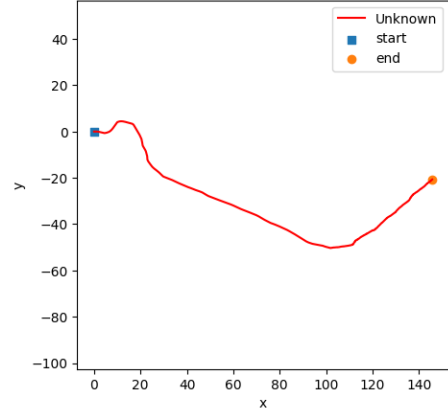


Fig. 1. Dataset00 Odometry generated from EKF prediction step

This odometry looks roughly correct, because it align with the correct orientation translation with video on the robot.

With initial landmark covariance  $10^{-4}$  and observe noise  $V = 0.1$ , I got the following result:

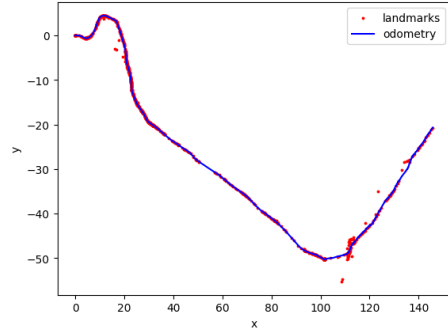


Fig. 2. Dataset00 landmarks

Then with predict noise  $W_v = 0.01$  and  $W_\omega = 10^{-3}$  and observe noise  $V = 10^3$ . I get the following result:

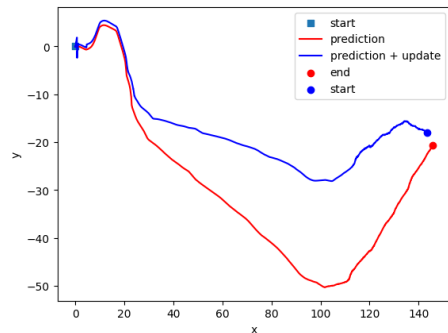


Fig. 3. Dataset00 result

The result of previous parameter looks inconsistent with the video, so I change  $V = 7.5 \times 10^3$ , and get a better result:

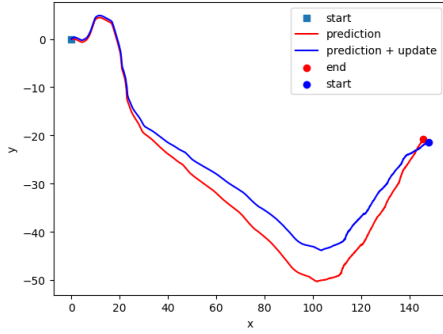


Fig. 4. Dataset00 result

### B. Dataset 01

The odometry generated from EKF prediction step:

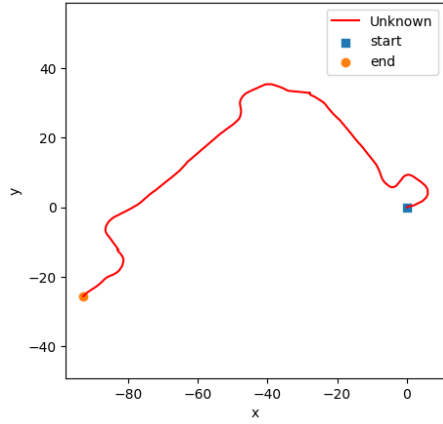


Fig. 5. Dataset01 Odometry generated from EKF prediction step

With initial landmark covariance  $10^{-4}$  and observe noise  $V = 0.1$ , I got the following result:

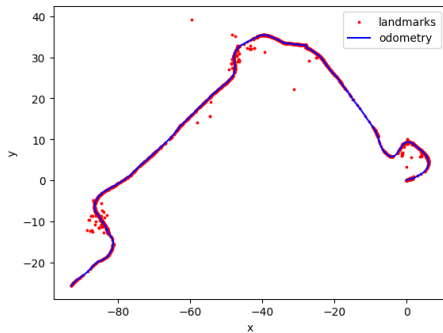


Fig. 6. Dataset01 landmarks

Then with predict noise  $W_v = 0.01$  and  $W_\omega = 10^{-3}$  and observe noise  $V = 10^3$ , I got the following result:

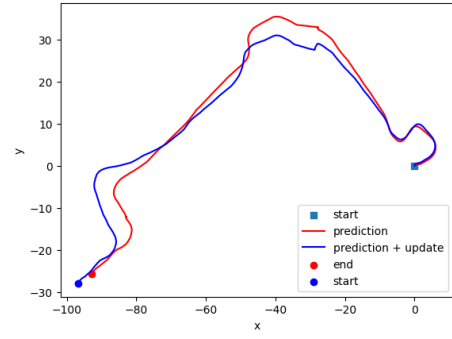


Fig. 7. Dataset01 result

### C. Dataset 02

The odometry generated from EKF prediction step:

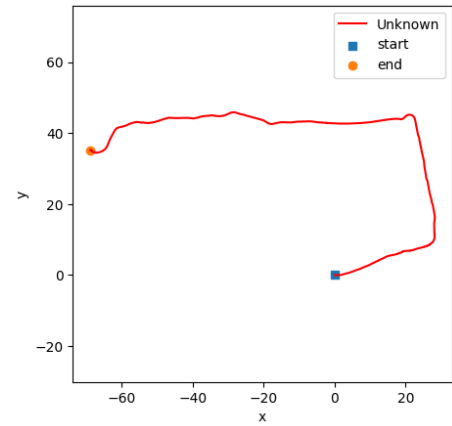


Fig. 8. Dataset02 Odometry generated from EKF prediction step

## V. OPTIONAL: FEATURE DETECTION AND MATCHING

### A. Technical approach

Use Opencv goodFeatureToTrack to extract the corner in image, then use Opencv calcOpticalFlowPyrLK to match features detect by left camera to right camera. This way, we could extract visual features from stereo camera images.

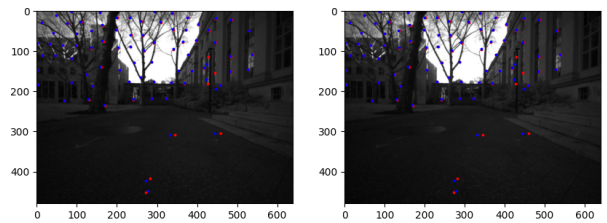


Fig. 9. Features in images from left and right camera (T = 1000)

Using calcOpticalFlowPyrLK, we can track features moving with time.

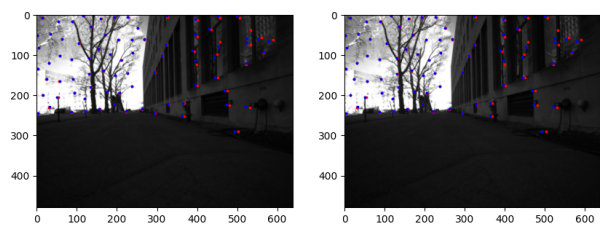


Fig. 10. Features in images from left and right camera ( $T = 2000$ )

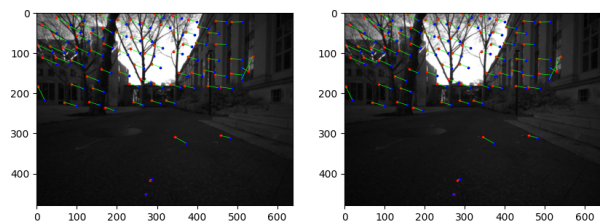


Fig. 11. Features in left camera with delta time = 15

## REFERENCES

- [1] Nikolay Atanasov, Sensing & Estimation in Robotics (Winter 2025), <https://natanaso.github.io/ece276a/schedule.html>.
- [2] scipy, <https://docs.scipy.org/doc/scipy/index.html>
- [3] opencv, <https://opencv.org/>